

Department of Information and Communication Technology

Faculty of Technology

University of Ruhuna

Object Oriented Programming Practicum

ICT 2132

Mini Project

2026 – B09

Group 5

Submitted to: Mr.P.H.P. Nuwan Laksiri

Submitted by: TG/2024/2095 - K.H.M. Hasaranga

TG/2024/2096 - N.V.S. Himansana

TG/2024/2080 - I.A.S. Sandeepani

TG/2024/2127 - D.S.R.N. Dahanayaka

Contents

Abstract 4
1. Introduction 5
1.1 Problem Statement 5
1.2 Proposed Solution 5
2. System Design and Architecture 6
2.1 System Architecture 7
2.2 Role-Based System View 8
2.3 Database Design 9
2.4 Entity Relationship (ER) Diagram 9
2.5 Class Diagram 11
2.6 Object-Oriented Concepts Applied 12
3. Methodology 13
3.1 Development Approach 13
3.2 Team Collaboration 13
3.3 Development Tools 13
4. Implementation (Selected Code) 14
4.1 User Hierarchy and Role Routing 14
4.2 Authentication 15
4.3 Database Access Layer 16
5. User Interface Screenshots 17
5.1 Login Screen 17
5.2 Administrator Screens 18
5.3 Lecturer Screens 20
5.4 Technical Officer Screens 22
5.5 Undergraduate Screens 23
6. Database Details and Query Results 25
6.1 Database Overview 25
6.2 Users and Accounts 26
6.3 Courses 26
6.4 Attendance and Medical Records 27
6.5 Marks, Eligibility and Grades 27
6.6 Database Security 28
7. Security 29
7.1 Security Requirements and Implementation 29

7.2 Additional Safeguards 29
8. Conclusion 30
8.1 Project Summary 30
8.2 Achievements 30
8.3 Limitations and Future Enhancements 30
9. Individual Contributions 31
TG/2024/2095 - K.H.M. Hasaranga (Group Leader) 31
9.2 TG/2024/2096 - N.V.S. Himansana 32
TG/2024/2080 - I.A.S.S. Appuhami 33
TG/2024/2127 - D.S.R.N. Dahanayaka 34
10. GitHub Project Link 35
11. References 36
12. Appendices 37
12.1 Hardware Requirements 37
12.2 Software Requirements 37
12.3 Setup and Run Instructions 37

Abstract

This project implements a Faculty Management System for the Faculty of Technology, University of Ruhuna, as a desktop application written in Java. The graphical interface is built with JavaFX (FXML and CSS) and all data is stored in a central MySQL database that is accessed through JDBC. The system replaces manual record keeping by managing user profiles, courses, undergraduate records, attendance, medical submissions, marks, grades, notices, timetables and course materials in one place.

Four roles are supported – Administrator, Lecturer, Technical Officer and Undergraduate – and each role is taken to its own dashboard after login and can only perform the operations that belong to it. The application calculates attendance percentages for theory, practical and combined sessions, identifies the five attendance scenarios required by the project brief, checks exam eligibility (attendance of at least 80% and CA marks of at least 40%), and produces grade points together with SGPA and CGPA for each undergraduate.

The code follows a layered architecture (GUI → Controller → Service → DAO → JDBC → MySQL) and demonstrates the main object-oriented concepts: classes and objects, inheritance, abstraction, polymorphism, encapsulation and exception handling. Security is addressed through BCrypt password hashing, role-based access control, input validation, parameterised SQL statements, a least-privilege database user and session clearing on logout.

1. Introduction

1.1 Problem Statement

Students, lecturers and staff of the Faculty of Technology need to access academic information every day, but there is no single place where each user can see the information that belongs to him or her. The main problems are:

- **Students cannot check their GPA.** There is no simple way for an undergraduate to see grades, grade points, SGPA and CGPA.
- **Lecture notes are hard to find.** There is no single place where students can find the course materials uploaded by their lecturers.
- **Students cannot view the timetable** of their courses, and notices do not reach the right students.
- **Lecturers cannot view their students' details.** A lecturer cannot easily see the profile, attendance, medical, CA marks and eligibility of the students of his or her courses.
- **Attendance, medical and marks records are scattered,** so checking whether an undergraduate is eligible for the final examination (attendance of at least 80% and CA marks of at least 40%) is slow and error-prone.
- **Different users need different views of the same data** (administrators, lecturers, technical officers and students), but access is not controlled by role.

There is therefore a need for one integrated application that gives every user the information and functions that their role needs, calculates attendance, eligibility, grades and GPA automatically, and protects passwords and personal data.

1.2 Proposed Solution

To solve these problems we developed the Faculty Management System, a JavaFX desktop application backed by a MySQL database that keeps everything in one place. Students can view their GPA, course materials, timetable, attendance and notices; lecturers can view their students' details; and every role works only with the data that belongs to it. The main parts of the solution are:

Multi-role access control

- **Administrator:** creates and maintains users, courses, notices and timetables, and can view attendance, marks and medical data.
- **Lecturer:** updates own profile (except username and password), manages course materials, uploads marks for all evaluation types, and views student details, eligibility, results, attendance, medicals and notices.
- **Technical Officer:** adds and maintains attendance and medical records, approves or rejects medicals, and views notices and the timetable of the department.
- **Undergraduate:** views own courses, attendance, medicals, grades and GPA, timetable and notices, and updates only contact details and profile picture.

Core functionalities

- Login with BCrypt-verified passwords and automatic routing to the dashboard of the user's role.
- Attendance for 15 sessions of every theory and practical component, with medical linking and batch/individual summaries.
- CA summary, exam eligibility, grades, SGPA and CGPA for individuals and the whole batch.

2. System Design and Architecture

2.1 System Architecture

The system is a standalone desktop application with a layered architecture. Each layer only talks to the layer below it, which keeps the interface independent from the database and makes the code easier to maintain.

Layer / package	Responsibility
controller	JavaFX controllers (LoginController and the dashboard + section controllers of every role). They read user input, call services and update the screen.
service	Business rules: authentication, role checks, mark validation, eligibility, attendance calculation, GPA, session handling.
dao	Data access interfaces and their JDBC implementations. All SQL lives here and uses PreparedStatement.
model	Plain Java classes: User → Staff → Admin / Lecturer / TechnicalOfficer, Undergraduate, Course, Attendance, MedicalRecord, MarkEntry, Notice, enums.
config, util, exception	DBConnection, PasswordUtil (BCrypt), Validator, FileStorageUtil, DataSeeder, and custom exceptions (AuthenticationException, ValidationException, ServiceException).
resources	FXML layouts, CSS style sheets and images for every dashboard.

2.2 Role-Based System View

After a successful login, the User object that was loaded from the database is asked for its own dashboard (getDashboardFxml()). Because every subclass overrides this method, the application does not need any role-specific if/else chain to choose the screen. Figure 2.2 shows the four role views and the sections available in each.

Role	Main permissions	Restrictions
Administrator	Create/update/deactivate users, courses, notices, timetables; view attendance, marks, medicals.	Highest access; still logs in through the same BCrypt check.
Lecturer	Materials, marks (0–100), eligibility, results, student details, attendance and medicals of own courses, notices, timetable.	Cannot change username/password; services accept only courses assigned to the lecturer.
Technical Officer	Add/maintain attendance and medicals, approve or reject medicals, view notices and department timetable.	No access to marks or user administration.
Undergraduate	View own courses, attendance, medicals, grades/GPA, timetable, notices.	May edit only contact details and profile picture.

2.3 Database Design

The database faculty_db has 16 tables and uses InnoDB with foreign keys, CHECK constraints and UNIQUE constraints. The shared user information is kept in the users table; Undergraduate and Staff are subtypes (shared primary key), and Admin, Lecturer and Technical Officer are subtypes of Staff. This mirrors the class hierarchy in the code. The most important tables are summarised below.

Table	Purpose	Key columns and rules
users	Login and personal data of every person	user_id (PK), username (UNIQUE), password_hash (BCrypt), role ENUM(ADMIN, LECTURER, TECHNICAL_OFFICER, UNDERGRADUATE), email (UNIQUE), is_active, profile_picture_path
undergraduate	Student-specific data	stu_id (PK, FK → users), reg_number (UNIQUE), batch, status ENUM(REGULAR, REPEAT, BATCH_MISSED), dep_id (FK)
course	Course details	course_id (PK), course_code (UNIQUE), credits_theory, credits_practical, semester, course_type, lecture_id (FK → lecturer), created_by (FK → admin)
enrollment	Student ↔ course (M:N)	stu_id, course_id, academic_year; UNIQUE(stu_id, course_id, academic_year) lets repeat students re-enrol
attendance	One row per student per session	stu_id, course_id, session_id CHECK 1–15, session_type, session_date, status (PRESENT/ABSENT), session_hours = 2.0, medical_id (FK), to_id (FK)
medical_record	Medical submissions	stu_id, session_date, document_path, approval_status ENUM(PENDING, APPROVED, REJECTED), approved_by (FK → technical_officer)
marks	Evaluation marks	stu_id, course_id, evaluation_type, mark_category ENUM(CA, FINAL), marks_value CHECK 0–100
grade	Final mark, grade and grade point	stu_id, course_id, final_mark, grade, grade_point CHECK 0–4, semester (used for SGPA/CGPA)

2.4 Entity Relationship (ER) Diagram

The EER diagram prepared by the group (Figure 2.3, next page) shows the User supertype with its Staff and Undergraduate subtypes, and the relationships between Department, Course, Attendance, Medical Record, Marks, Grade, Notice and Timetable. The physical schema in db/schema.sql was derived directly from this diagram; a few columns that the application needs (for example profile_picture_path, is_active, reg_number, file_path and mark_category) were added during implementation.

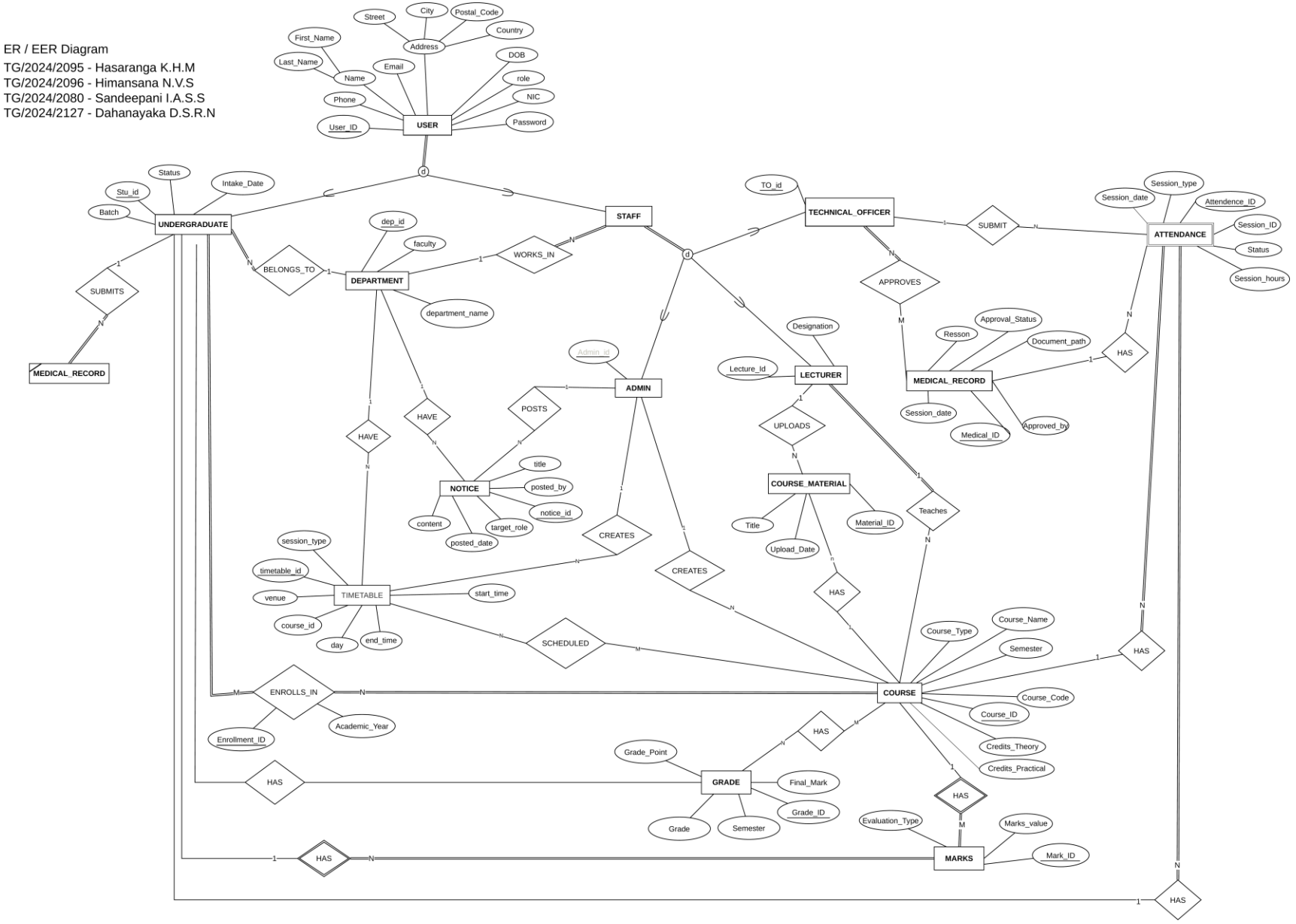


Figure 2.3 – ER / EER diagram of the Faculty Management System (faculty_db)

2.5 Class Diagram

Figure 2.4 shows the domain classes of the application and Figure 2.5 shows how the controller, service and DAO classes cooperate. Both diagrams were drawn from the final source code on the dev branch.

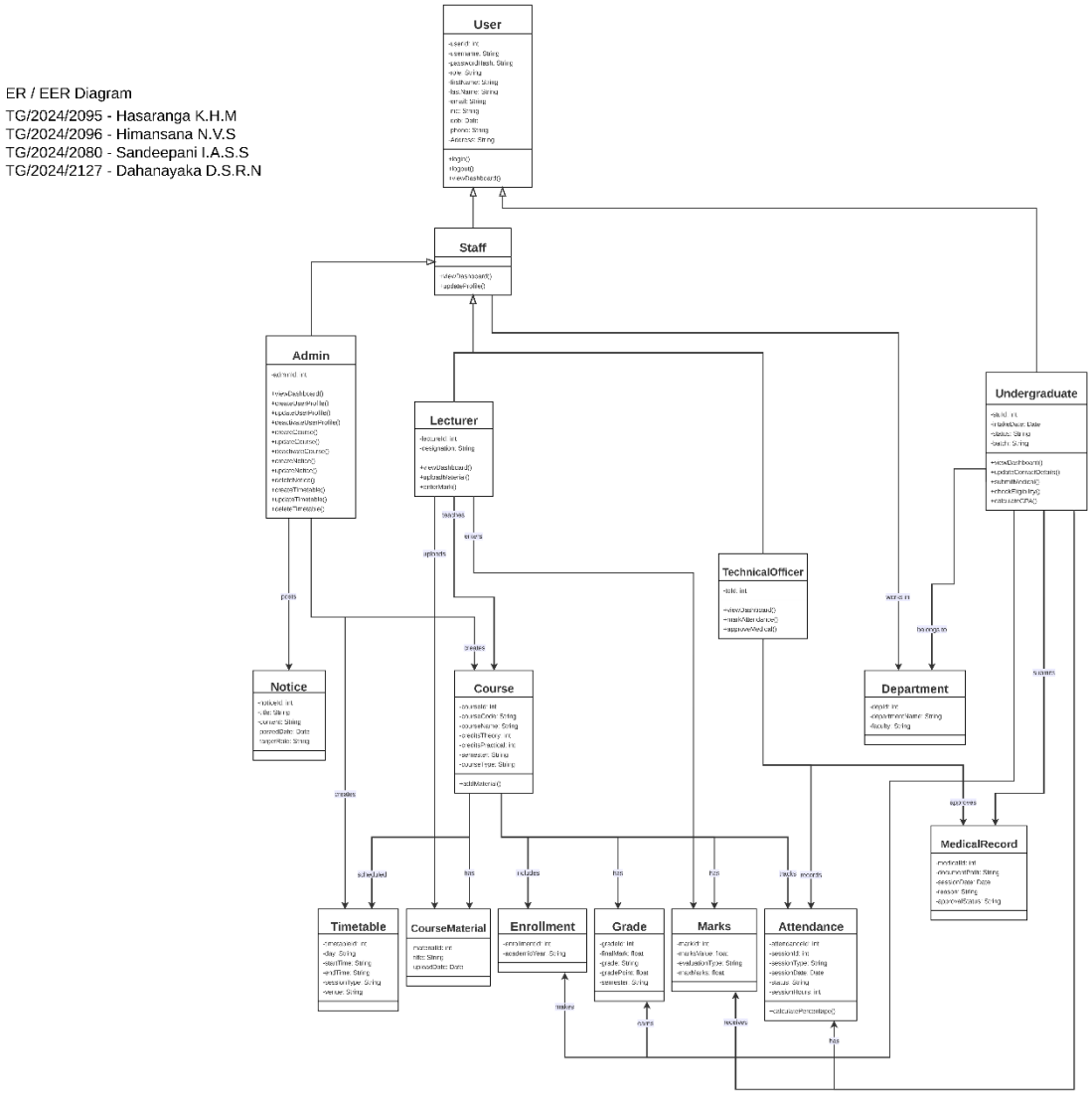


Figure 2.4 – Domain class diagram (model classes and relationships)

2.6 Object-Oriented Concepts Applied

Concept	Where it is used
Classes and Objects	Model classes such as Undergraduate, Lecturer, Course, Attendance and MedicalRecord; each database row is mapped to an object by the DAO layer.
Inheritance	User ← Staff ← Admin / Lecturer / TechnicalOfficer, and User ← Undergraduate. Shared fields and methods are written once in User.
Abstraction	User and Staff are abstract; DAO interfaces (UserDAO, LecturerMarksDAO, MedicalDAO, ...) hide the JDBC details from services and controllers.
Polymorphism	getDashboardFxml() and getDashboardTitle() are overridden in every role class, so the login code treats all users through the User type.
Encapsulation	All model fields are private with getters/setters; DBConnection, PasswordUtil and AttendanceCalculator are final utility classes with private constructors.
Exception handling	AuthenticationException, ValidationException and ServiceException; SQLException is caught in DAO/service code and converted to friendly messages; SecurityException is thrown for forbidden operations.
Database handling	JDBC with try-with-resources, PreparedStatement and a central DBConnection.
GUI	JavaFX scenes defined in FXML, styled with CSS and driven by controller classes.

3. Methodology

3.1 Development Approach

The project followed an iterative approach that matched the deadlines of the course unit:

Phase	Period	Work done
Planning and requirements	Early Sep 2026	Project brief analysed; repository and branches created; SRS written (submitted 13 Sep).
Design	Mid Sep 2026	EER diagram, class diagram and MySQL schema prepared (submitted 20 Sep).
Core development	Sep – Oct 2026	Database layer, login and role routing, user hierarchy, admin dashboard.
Role modules	Oct 2026	Lecturer, Technical Officer and Student modules built in parallel on separate branches.
Integration	Mid – late Oct 2026	Pull requests reviewed and merged into the dev branch; seed data for all attendance scenarios added.
Refinement and documentation	Late Oct 2026	Bug fixes, UI polish, final report (due 1 Nov).

3.2 Team Collaboration

The team used Git and GitHub. Every member worked on a personal branch (TG2095_Hasaranga, TG2096_Himansana, TG2127_Sandaru and TG2080_Sudeshika). Finished work was merged into the shared dev branch through pull requests that were reviewed and merged by the group leader, who also resolved merge conflicts. The history of the dev branch contains 33 merged pull requests between 6 September and 30 October 2026.

3.3 Development Tools

- **Language:** Java 17 (OpenJDK)
- **GUI:** JavaFX 21.0.2 with FXML and CSS
- **Database:** MySQL 8.0+ with mysql-connector-j 8.4.0 (JDBC); XAMPP or Docker for local servers
- **Security library:** jBCrypt 0.4
- **Build tool:** Apache Maven (javafx-maven-plugin 0.0.8)
- **IDE:** IntelliJ IDEA;
- **Version control:** Git and GitHub;
- **Diagrams:** draw.io, Graphviz
- **Documentation:** Microsoft Word, PDF

4. Implementation (Selected Code)

This chapter shows short, simplified extracts from the source code. Comments and unrelated lines have been trimmed to keep the extracts readable.

4.1 User Hierarchy and Role Routing

User is an abstract base class. Each role overrides the two abstract methods, which is how the login screen finds the correct dashboard without any role-specific conditions.

```
public abstract class User {
    private int userId;
    private String username;
    private String passwordHash;
    private UserRole role;
    // ... other private fields with getters/setters

    public abstract String getDashboardFxml();   // polymorphism
    public abstract String getDashboardTitle();

    public String getFullName() {
        return firstName + " " + lastName;
    }
}
```

```
public class Lecturer extends Staff {
    private String designation;

    @Override
    public String getDashboardFxml() {
        return "/fxml/lecture/lecturer_dashboard.fxml";
    }
    @Override
    public String getDashboardTitle() {
        return "Lecturer Dashboard";
    }
}
```

4.2 Authentication

AuthService verifies the password with BCrypt. The same error message is returned for an unknown user and a wrong password so that an attacker learns nothing, and deactivated accounts cannot log in.

```
public User login(String username, String password)
        throws AuthenticationException, SQLException {
    User user = userDAO.findByUsername(username);

    if (user == null
            || !PasswordUtil.verify(password, user.getPasswordHash())) {
        throw new AuthenticationException(
            "Invalid username or password.");
    }
    if (!user.isActive()) {
        throw new AuthenticationException(
            "This account is deactivated.");
    }
    return user;
}
```

```
// PasswordUtil
public static String hash(String plain) {
    return BCrypt.hashpw(plain, BCrypt.gensalt(12));
}
```

4.3 Database Access Layer

All SQL is written in DAO classes with ? placeholders. The DAO also builds the correct subclass of User according to the role column, which completes the polymorphic design.

```
public User findByUsername(String username) throws SQLException {
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps =
             con.prepareStatement(FIND_BY_USERNAME)) {
        ps.setString(1, username);     // ... WHERE u.username = ?
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? mapRow(rs) : null;
        }
    }
}
```

```
switch (role) {                        // inside mapRow()
    case ADMIN             -> user = new Admin();
    case LECTURER          -> user = new Lecturer();
    case TECHNICAL_OFFICER -> user = new TechnicalOfficer();
    default                -> user = new Undergraduate();
}
```

5. User Interface Screenshots

This chapter shows the graphical user interface of the Faculty Management System for every role. All screenshots were taken from the running application using the demo data in db/seed_data.sql.

5.1 Login Screen

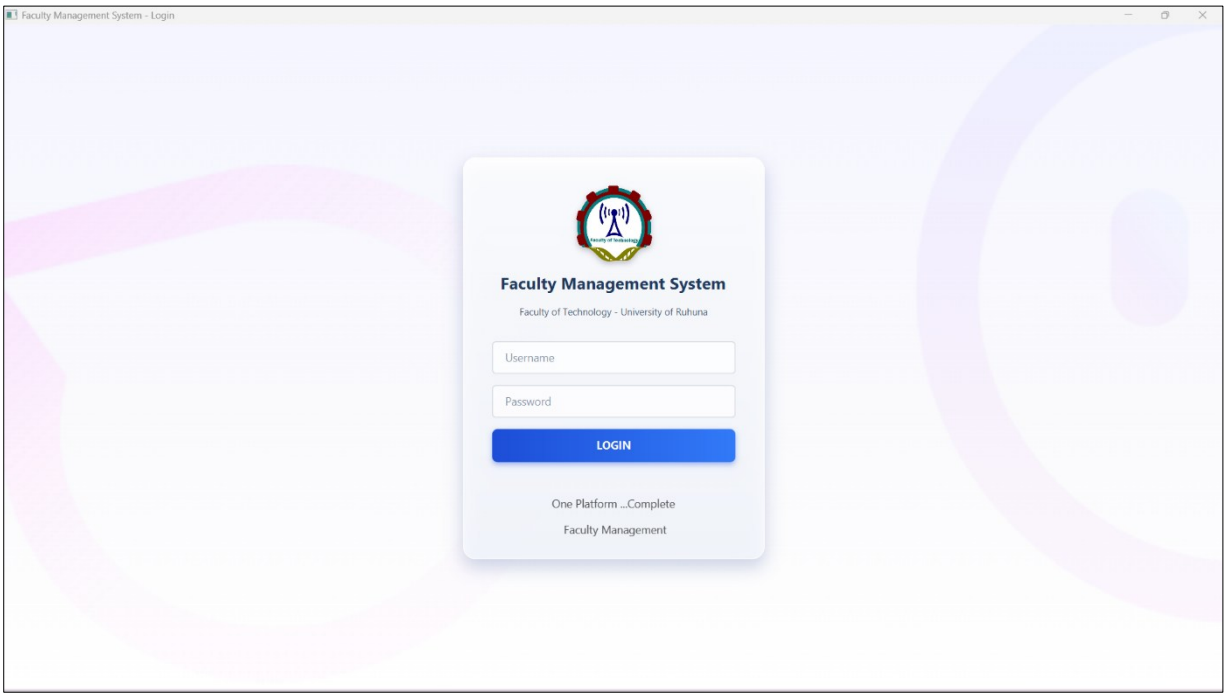


Figure 5.1 – Login screen

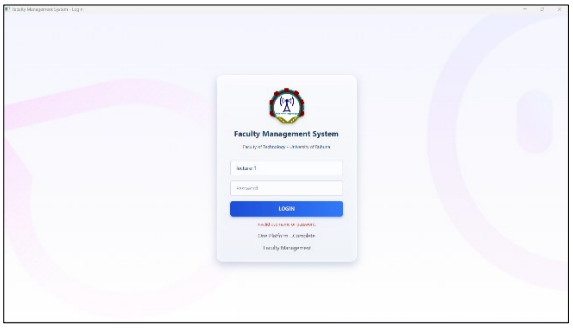


Figure 5.2 – Login error message for a wrong password

5.2 Administrator Screens

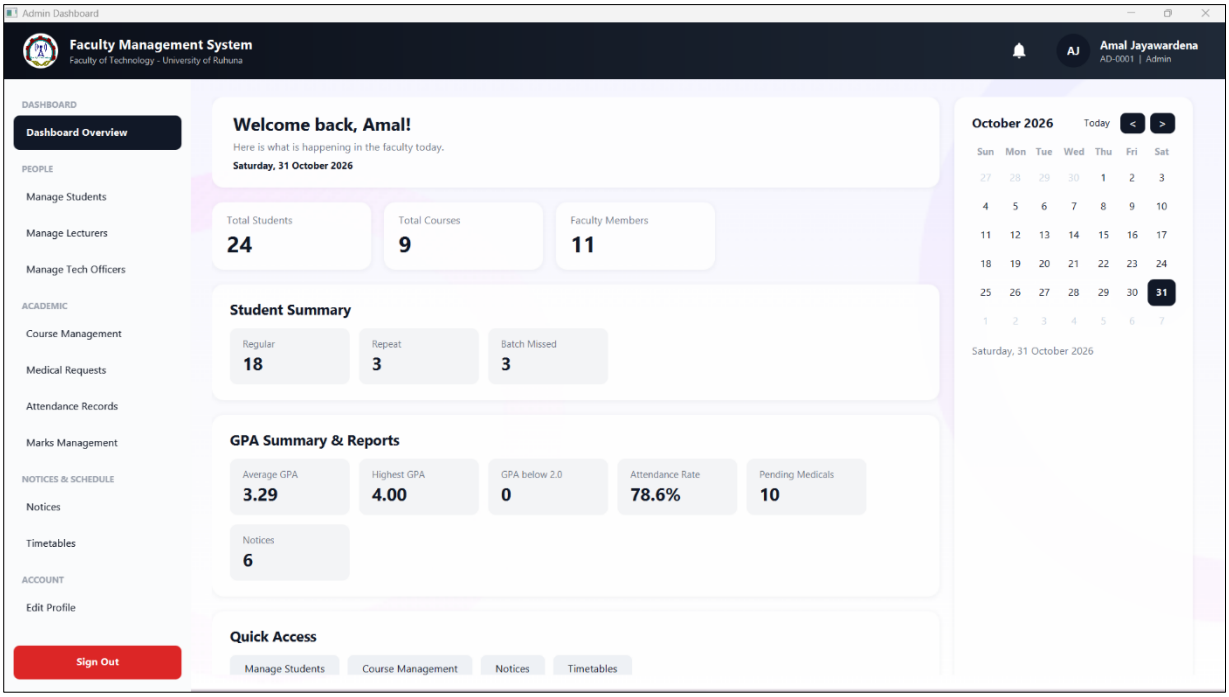


Figure 5.3 – Admin dashboard (home)

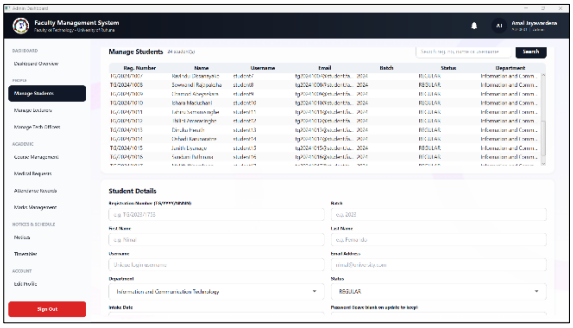


Figure 5.4 – Students section (list and add/edit form)

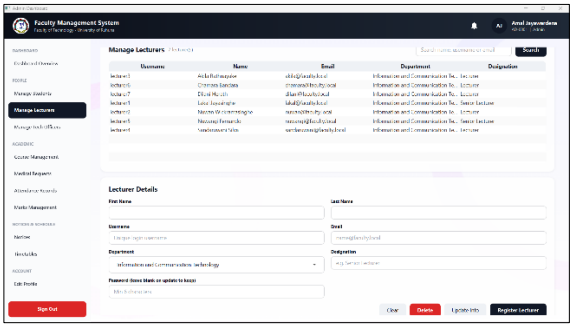


Figure 5.5 – Lecturers section

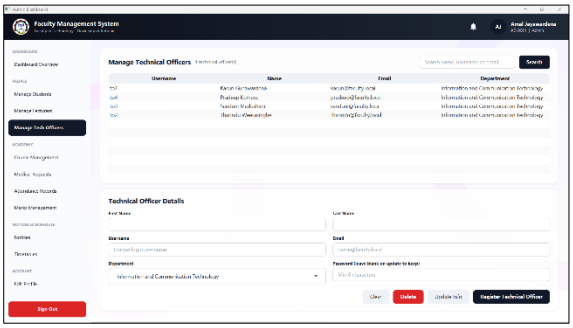


Figure 5.6 – Technical Officers section

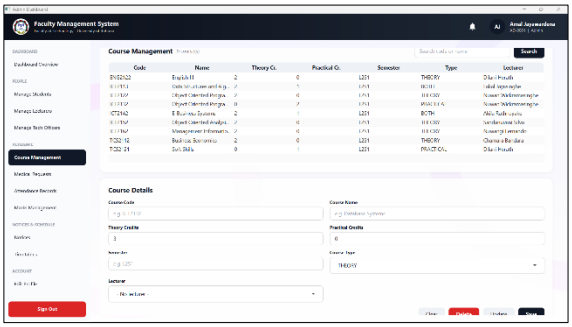


Figure 5.7 – Courses section

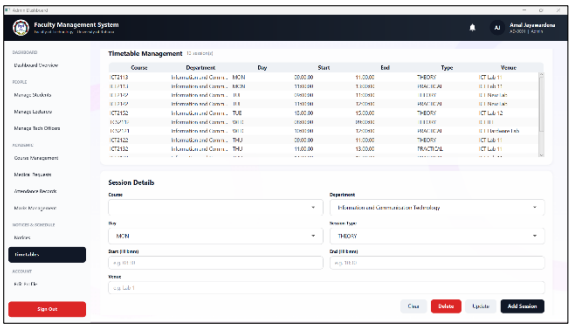


Figure 5.8 – Timetables section

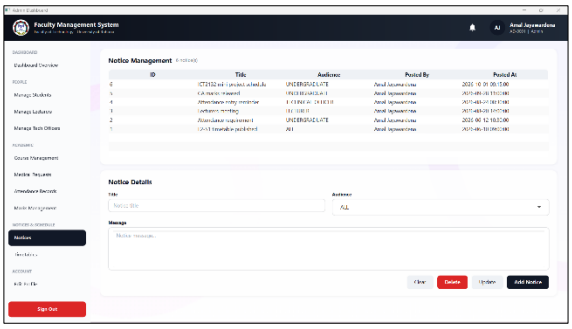


Figure 5.9 – Notices section

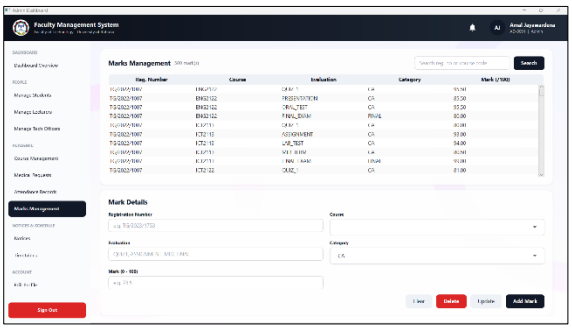


Figure 5.10 – Marks view

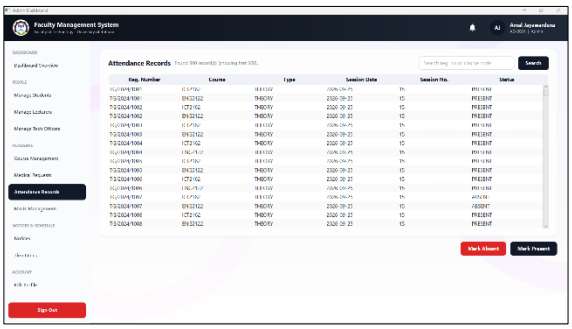


Figure 5.11 – Attendance view

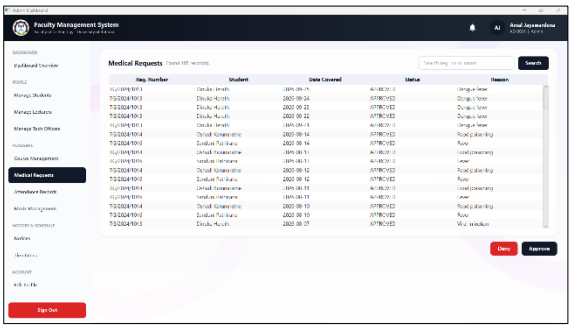


Figure 5.12 – Medical view

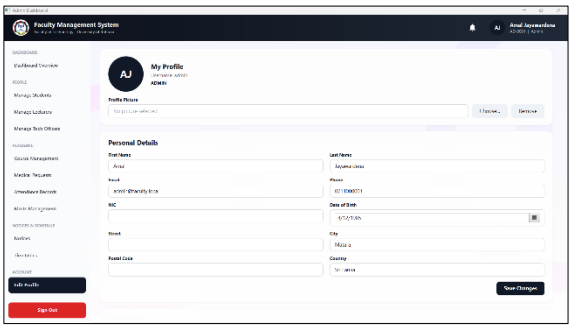


Figure 5.13 – Admin profile

5.3 Lecturer Screens

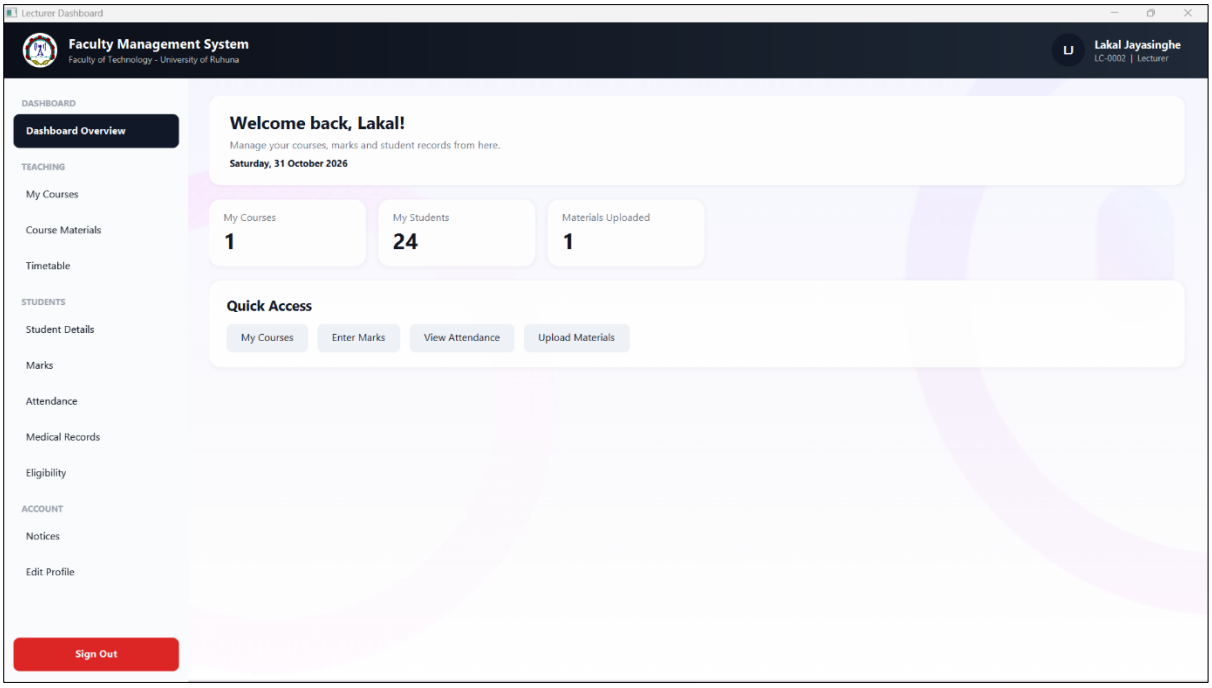


Figure 5.14 – Lecturer dashboard (home)

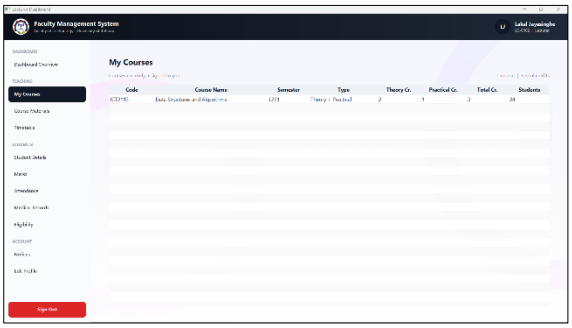


Figure 5.15 – My Courses

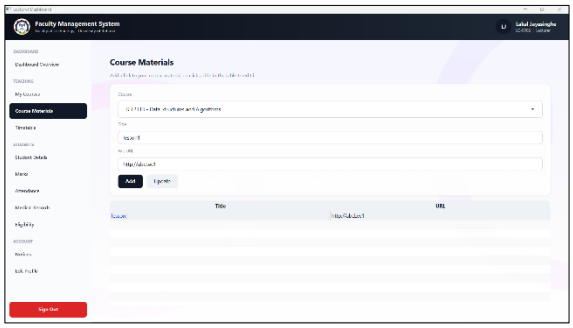


Figure 5.16 – Course Materials (upload / modify)

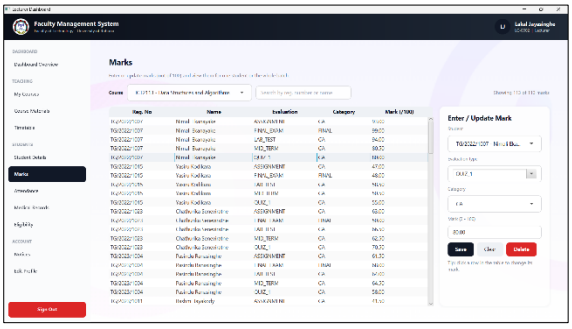


Figure 5.17 – Marks entry (adding a mark)

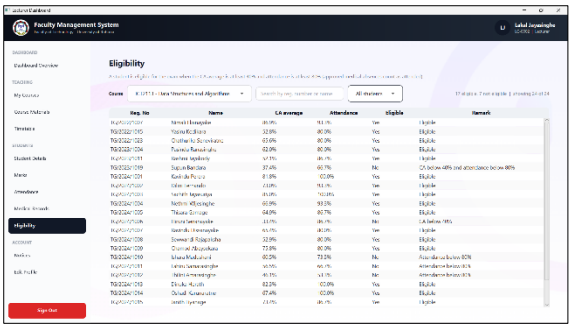


Figure 5.19 – Exam Eligibility (batch view)

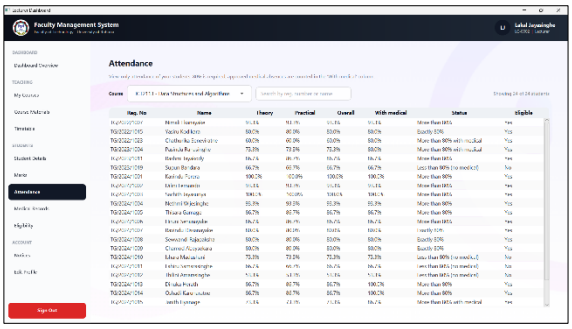


Figure 5.20 – Attendance summary (batch and individual)

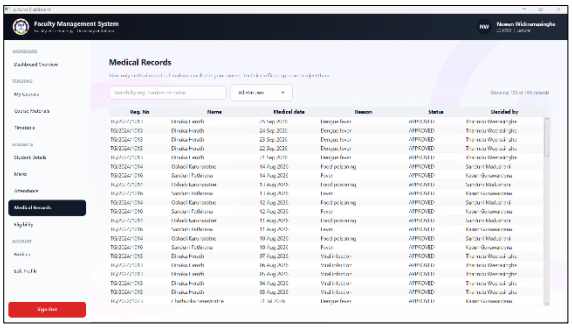


Figure 5.21 – Medical records view

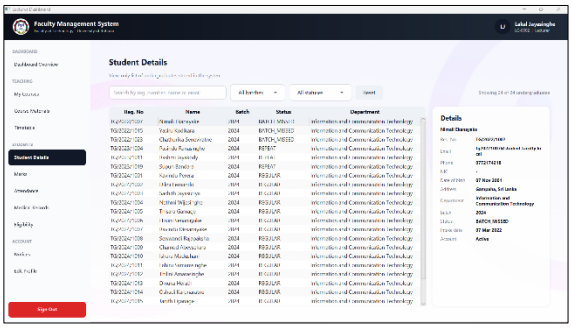


Figure 5.22 – Student details

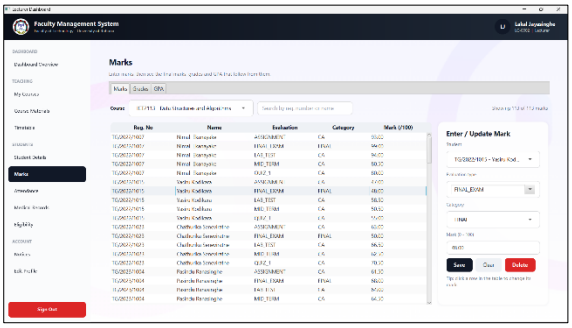


Figure 5.23 – Results – grades and GPA

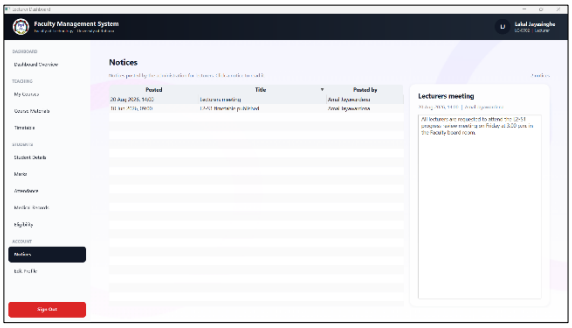


Figure 5.24 – Notices

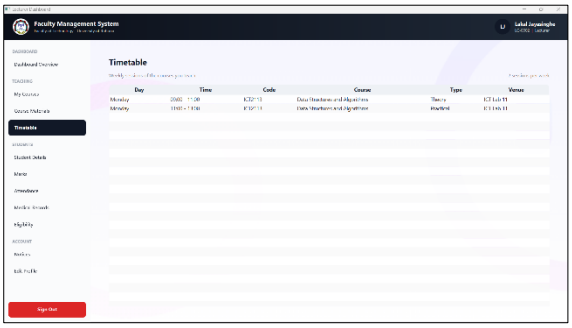


Figure 5.25 – Timetable

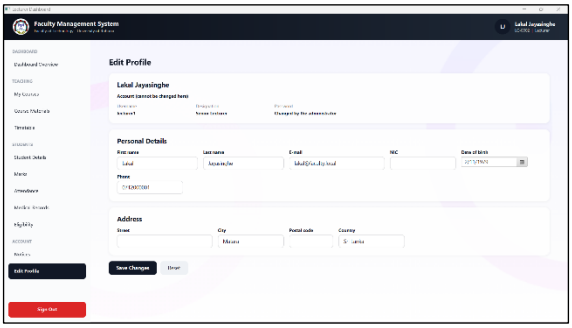


Figure 5.26 – Lecturer profile

5.4 Technical Officer Screens

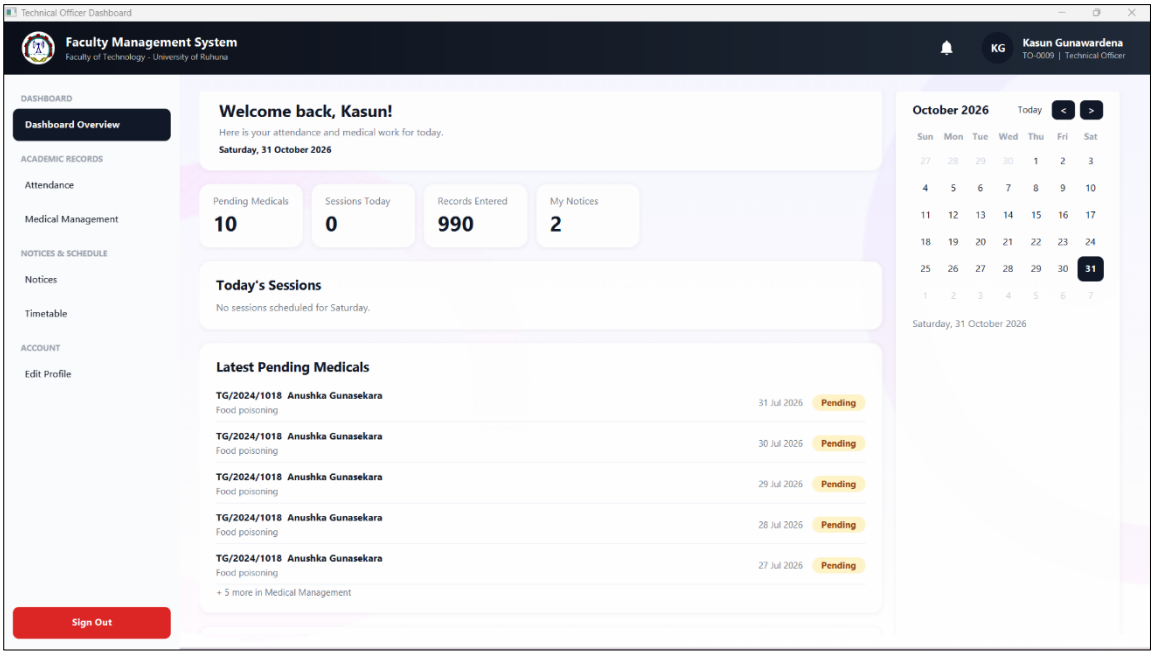


Figure 5.27 – Technical Officer dashboard (home)

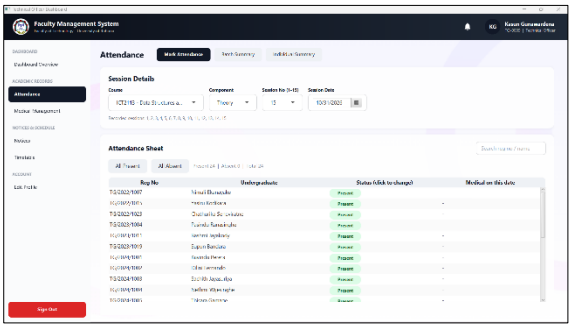


Figure 5.28 – Attendance sheet

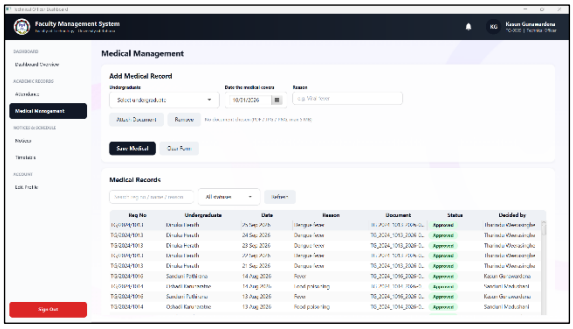


Figure 5.29 – Medical records – approve / reject

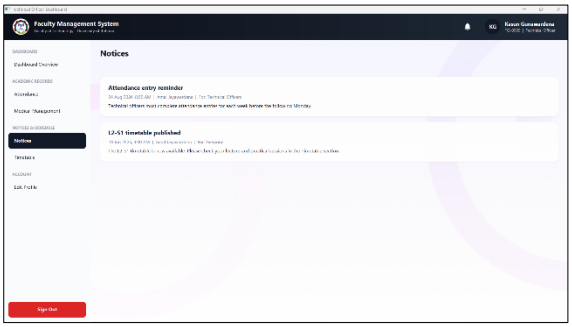


Figure 5.30 – Notices

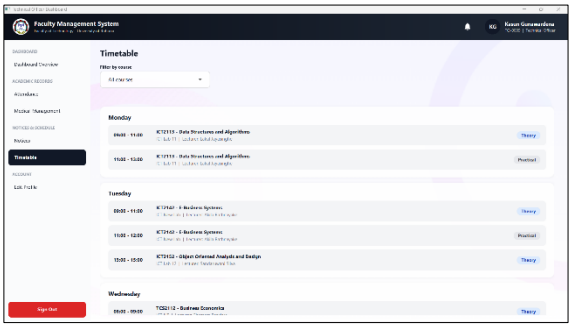


Figure 5.31 – Department timetable

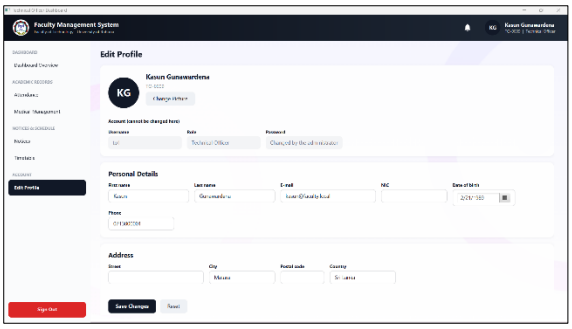


Figure 5.32 – Technical Officer profile

5.5 Undergraduate Screens

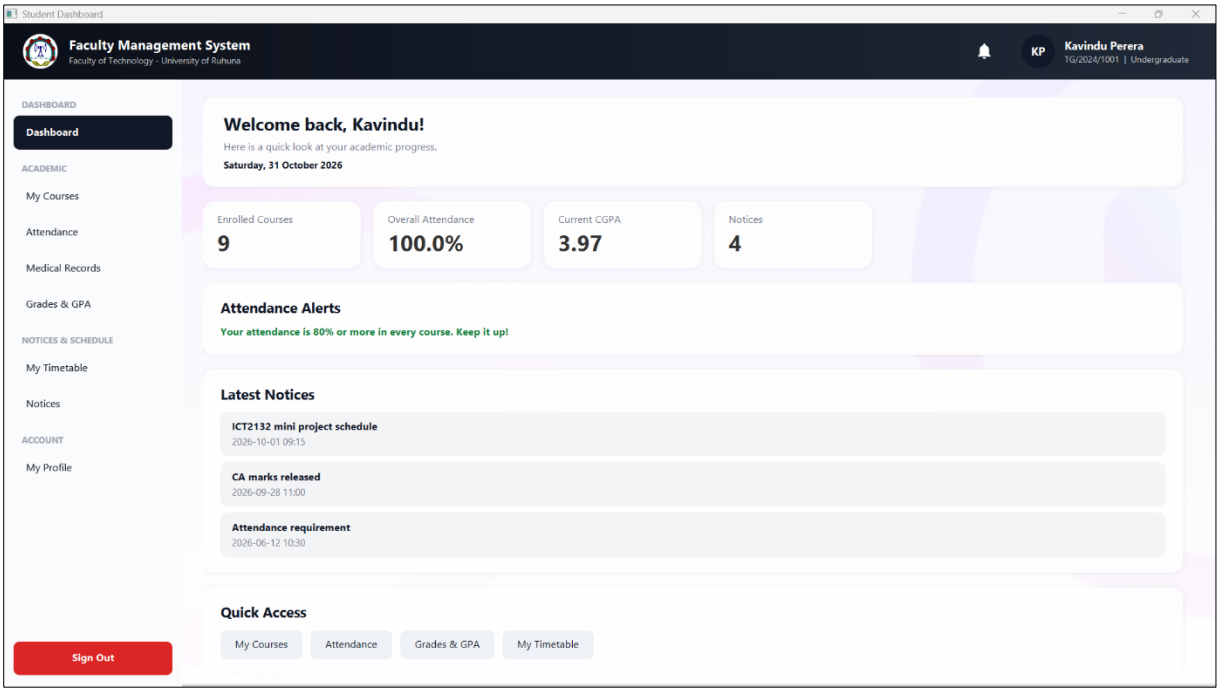


Figure 5.33 – Student dashboard (home)

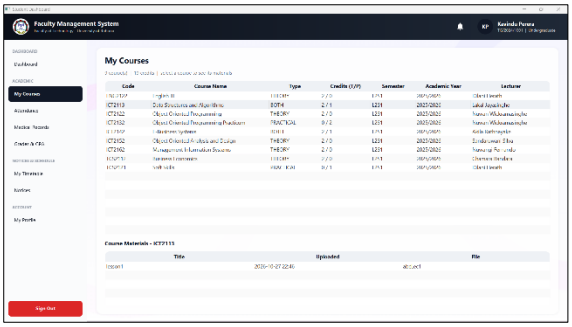


Figure 5.34 – My Courses

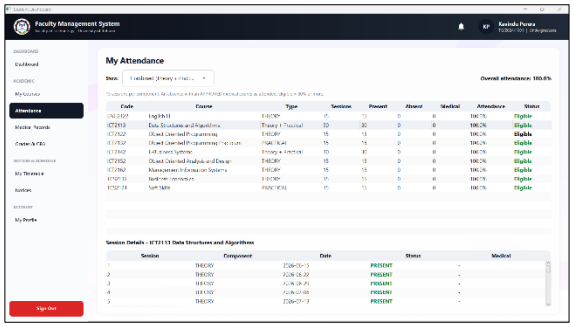


Figure 5.35 – Attendance (theory / practical / combined)

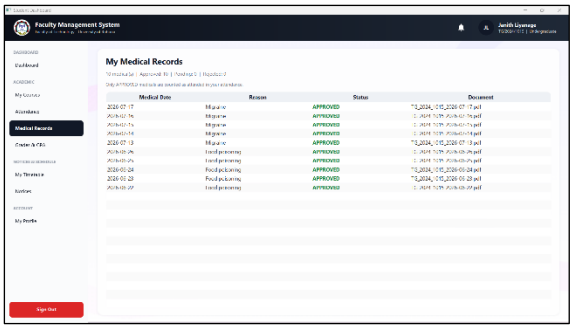


Figure 5.36 – Medical records and approval status

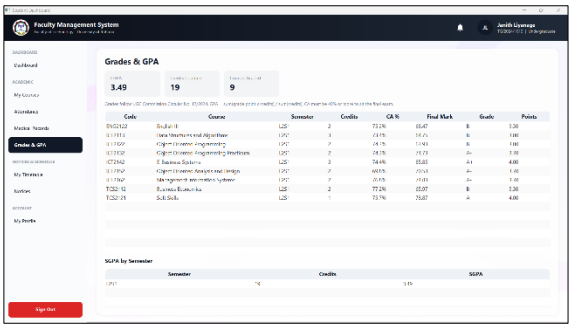


Figure 5.37 – Grades, SGPA and CGPA

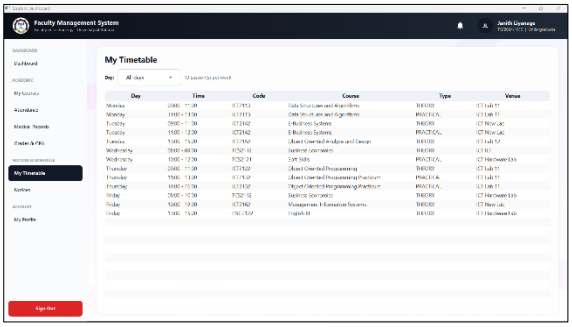


Figure 5.38 – Timetable

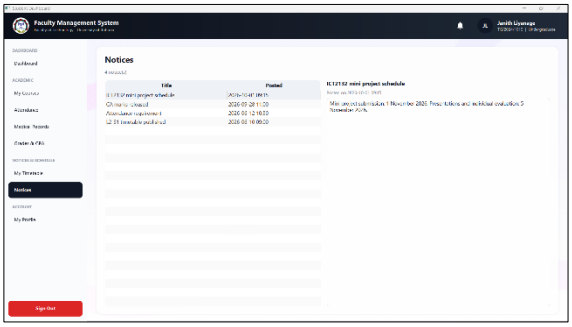


Figure 5.39 – Notices

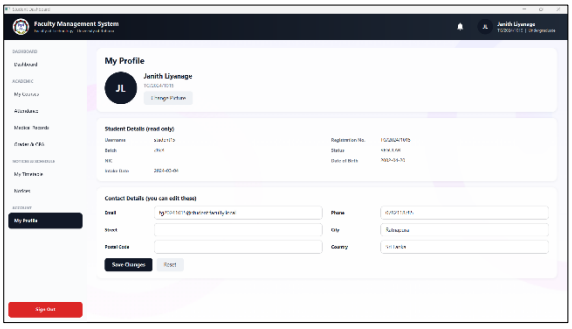


Figure 5.40 – Profile (contact details and picture only)

6. Database Details and Query Results

This chapter presents the faculty_db database as it exists after running db/schema.sql and db/seed_data.sql. Each query was executed in MySQL (Workbench, phpMyAdmin or the mysql client) and the result is shown below the query.

6.1 Database Overview

```
+----------------------+
| Tables_in_faculty_db |
+----------------------+
| admin                |
| attendance           |
| course               |
| course_material      |
| department           |
| enrollment           |
| grade                |
| lecturer             |
| marks                |
| medical_record       |
| notice               |
| staff                |
| technical_officer    |
| timetable            |
| undergraduate        |
| users                |
+----------------------+
```

Figure 6.1 – Tables of faculty_db

```
+-------------------+--------+------------+
| TABLE_NAME        | ENGINE | TABLE_ROWS |
+-------------------+--------+------------+
| admin             | InnoDB |          0 |
| attendance        | InnoDB |       3960 |
| course            | InnoDB |          9 |
| course_material   | InnoDB |          0 |
| department        | InnoDB |          0 |
| enrollment        | InnoDB |        216 |
| grade             | InnoDB |        157 |
| lecturer          | InnoDB |          7 |
| marks             | InnoDB |        853 |
| medical_record    | InnoDB |        105 |
| notice            | InnoDB |          6 |
| staff             | InnoDB |         12 |
| technical_officer | InnoDB |          4 |
| timetable         | InnoDB |         13 |
| undergraduate     | InnoDB |         24 |
| users             | InnoDB |         36 |
+-------------------+--------+------------+
```

Figure 6.2 – Database diagram from the MySQL tool

Query Q1: Row count of the main tables

Shows how much demo data is stored in each important table.

```
+----------------+------+
| tbl            | n    |
+----------------+------+
| users          |   36 |
| undergraduate  |   24 |
| course         |    9 |
| enrollment     |  216 |
| attendance     | 3960 |
| medical_record |  106 |
| marks          |  853 |
| grade          |  157 |
| notice         |    6 |
| timetable      |   13 |
+----------------+------+
```

Result 1 – Row count of the main tables

6.2 Users and Accounts

Query Q2: Users per role

Confirms the required data: 1 admin, at least 5 lecturers, at least 4 technical officers and at least 20 undergraduates.

role	total
ADMIN	1
LECTURER	7
TECHNICAL_OFFICER	4
UNDERGRADUATE	24

Result 2 – Users per role

Query Q3: Undergraduates by status

Shows that regular, repeat and batch-missed students are stored.

status	total
REGULAR	18
REPEAT	3
BATCH_MISSED	3

Result 3 – Undergraduates by status

Query Q4: Password hashes (BCrypt)

Proves that passwords are stored only as BCrypt hashes ($2a$ / $2b$ prefix, 60 characters), never as plain text (SEC-01).

username	role	hash_prefix	hash_length
admin	ADMIN	$2a$12$	60
lecturer1	LECTURER	$2a$12$	60
lecturer2	LECTURER	$2a$12$	60
lecturer3	LECTURER	$2a$12$	60
lecturer4	LECTURER	$2a$12$	60
lecturer5	LECTURER	$2a$12$	60
lecturer6	LECTURER	$2a$12$	60
lecturer7	LECTURER	$2a$12$	60

Result 4 – Password hashes (BCrypt)

6.3 Courses

Query Q5: Courses of the semester

Lists the Level 2 Semester 1 courses with theory and practical credits.

course_code	course_name	credits_theory	credits_practical	semester	course_type
TCS2112	Business Economics	2	0	L2S1	THEORY
ICT2113	Data Structures and Algorithms	2	1	L2S1	BOTH
ICT2122	Object Oriented Programming	2	0	L2S1	THEORY
ICT2132	Object Oriented Programming Practicum	0	2	L2S1	PRACTICAL
ICT2142	E-Business Systems	2	1	L2S1	BOTH
ICT2152	Object Oriented Analysis and Design	2	0	L2S1	THEORY
ICT2162	Management Information Systems	2	0	L2S1	THEORY
ENG2122	English III	2	0	L2S1	THEORY
TCS2121	Soft Skills	0	1	L2S1	PRACTICAL

Result 5 – Courses of the semester

6.4 Attendance and Medical Records

Query Q6: Medical records by approval status

Shows that medicals can be pending, approved or not approved.

approval_status	total
APPROVED	85
REJECTED	10
PENDING	11

Result 6 – Medical records by approval status

Query Q7: Attendance scenarios for one course

Shows raw attendance and attendance including approved medicals. Together they cover the five scenarios: more than 80%, exactly 80%, less than 80% without medical, more than 80% with medical and less than 80% with medical. Change course_id to check another course.

reg_number	status	raw_pct	approved_medical_absences	effective_pct
TG/2024/1001	REGULAR	100.0	0	100.0
TG/2024/1003	REGULAR	100.0	0	100.0
TG/2024/1002	REGULAR	93.3	0	93.3
TG/2024/1004	REGULAR	93.3	0	93.3
TG/2022/1007	BATCH_MISSED	93.3	0	93.3
TG/2024/1005	REGULAR	86.7	0	86.7
TG/2024/1006	REGULAR	86.7	0	86.7
TG/2024/1013	REGULAR	86.7	2	100.0
TG/2024/1014	REGULAR	86.7	2	100.0
TG/2023/1011	REPEAT	86.7	0	86.7
TG/2024/1007	REGULAR	80.0	0	80.0
TG/2024/1008	REGULAR	80.0	0	80.0
TG/2024/1009	REGULAR	80.0	0	80.0
TG/2022/1015	BATCH_MISSED	80.0	0	80.0
TG/2024/1010	REGULAR	73.3	0	73.3
TG/2024/1015	REGULAR	73.3	2	86.7
TG/2024/1016	REGULAR	73.3	3	93.3
TG/2023/1004	REPEAT	73.3	2	86.7
TG/2024/1011	REGULAR	66.7	0	66.7
TG/2023/1019	REPEAT	66.7	0	66.7
TG/2024/1017	REGULAR	60.0	2	73.3
TG/2022/1023	BATCH_MISSED	60.0	3	80.0
TG/2024/1012	REGULAR	53.3	0	53.3
TG/2024/1018	REGULAR	53.3	2	66.7

Result 7 – Attendance scenarios for one course

6.5 Marks, Eligibility and Grades

Query Q8: CA average and CA eligibility

CA part must be 40% or more to sit the final exam. Change course_id to check another course.

reg_number	course_code	ca_avg	ca_status
TG/2023/1019	TCS2112	39.83	CA FAIL
TG/2022/1015	TCS2112	42.67	CA OK
TG/2024/1012	TCS2112	44.00	CA OK
TG/2024/1017	TCS2112	50.83	CA OK
TG/2024/1018	TCS2112	51.00	CA OK
TG/2024/1011	TCS2112	53.67	CA OK
TG/2023/1011	TCS2112	53.83	CA OK
TG/2024/1008	TCS2112	58.00	CA OK
TG/2024/1006	TCS2112	58.83	CA OK
TG/2024/1010	TCS2112	61.33	CA OK
TG/2024/1016	TCS2112	62.67	CA OK
TG/2024/1005	TCS2112	63.50	CA OK
TG/2024/1007	TCS2112	68.00	CA OK
TG/2024/1014	TCS2112	68.83	CA OK

Result 8 – CA average and CA eligibility

Query Q9: Grade distribution

Number of students for each grade (UGC Circular 12-2024 scale).

grade	total
A+	22
A	28
A-	24
B+	20
B	20
B-	23
C+	11
C	8
D+	1

Result 9 – Grade distribution

Query Q10: SGPA per student and semester

Credit-weighted SGPA calculated from grade points and course credits.

reg_number	semester	sgpa
TG/2024/1003	L2S1	4.00
TG/2022/1007	L2S1	4.00
TG/2024/1001	L2S1	3.97
TG/2024/1013	L2S1	3.97
TG/2024/1002	L2S1	3.83
TG/2024/1009	L2S1	3.65
TG/2024/1015	L2S1	3.49
TG/2024/1014	L2S1	3.48
TG/2022/1023	L2S1	3.47
TG/2024/1004	L2S1	3.34
TG/2023/1004	L2S1	3.25
TG/2024/1007	L2S1	3.18
TG/2024/1016	L2S1	3.13
TG/2024/1005	L2S1	3.02
TG/2024/1006	L2S1	2.65
TG/2023/1011	L2S1	2.51
TG/2024/1008	L2S1	2.26
TG/2022/1015	L2S1	2.06

Result 10 – SGPA per student and semester

6.6 Database Security

Query Q11: Privileges of the application user

Shows that the application database user has only SELECT, INSERT, UPDATE and DELETE (SEC-05). Run db/create_app_user.sql first.

Grants for faculty_app@localhost
GRANT USAGE ON *.* TO `faculty_app`@`localhost`
GRANT SELECT, INSERT, UPDATE, DELETE ON `faculty_db`.* TO `faculty_app`@`localhost`

Result 11 – Privileges of the application user

7. Security

Security requirements SEC-01 to SEC-08 were defined in the SRS. The table shows how each one is handled in the implementation.

7.1 Security Requirements and Implementation

ID	Requirement	Implementation
SEC-01	One-way password hashing	PasswordUtil uses jBCrypt (cost factor 12). Only password_hash is stored; plaintext passwords are never saved or logged. Seed data is inserted as BCrypt hashes.
SEC-02 **SEC-07**	Role-based access control re-checked in the service layer	The logged-in user is routed to a role dashboard. The Lecturer services call requireLecturer() (instanceof Lecturer via SessionManager) and requireMyCourse(), and throw SecurityException otherwise, so hiding a button is not the only protection.
SEC-03	Input validation	Validator checks required fields, length, e-mail, Sri Lankan phone numbers and old/new NIC formats; marks must be numeric and between 0 and 100; the database adds CHECK, UNIQUE and ENUM constraints as a second line of defence.
SEC-04	Parameterised SQL	All DAO queries use PreparedStatement with ? placeholders. The only plain Statement calls are fixed dashboard count queries that contain no user input.
SEC-05	Dedicated least-privilege DB user	db/create_app_user.sql creates faculty_app with SELECT, INSERT, UPDATE and DELETE only (no DROP, ALTER or CREATE).
SEC-06	Session kept only while logged in	SessionManager holds the current user and logout() calls SessionManager.clear().
SEC-08	Credentials outside source code	DBConnection reads the URL, user and password from db/db.properties at run time; nothing is hard-coded in Java classes.

7.2 Additional Safeguards

- A failed login never reveals whether the username or the password was wrong.
- Deactivated accounts (users.is_active = false) are refused at login (FR-05).
- Foreign keys with ON DELETE rules and UNIQUE keys prevent orphaned or duplicate records (for example one attendance row per student, course, session type and date).
- Uploaded medical documents and profile pictures are stored by FileStorageUtil in a separate uploads folder, and uploads/medicals is excluded from Git.
- Technical details of database errors are written to the console for developers, while users only see a friendly message.

8. Conclusion

8.1 Project Summary

The Faculty Management System fulfils the requirements of the ICT2132 mini project. It combines a JavaFX interface, a layered Java code base and a normalised MySQL database to handle user profiles, courses, attendance, medicals, marks, grades, notices, timetables and course materials for four user roles. The code base demonstrates the object-oriented concepts requested in the project brief and applies the security requirements that were defined in the SRS.

8.2 Achievements

- Complete multi-role desktop application with four dashboards and 25+ functional screens.
- Database with 16 tables, constraints and seed data: 1 admin, 7 lecturers, 4 technical officers and 24 undergraduates (18 regular, 3 repeat, 3 batch-missed).
- Attendance data for all 15 sessions of every theory and practical component, covering all five required scenarios, with approved, rejected and pending medicals.
- Automatic calculation of attendance, CA average, eligibility, grade points, SGPA and CGPA.
- BCrypt hashing, role checks, validation, prepared statements and a least-privilege database user.
- Team workflow with personal branches and reviewed pull requests; SRS, EER and class diagrams completed on time.

8.3 Limitations and Future Enhancements

- Add role checks in the Technical Officer and Student service classes in the same way as in the Lecturer services.
- Calculate final marks and grades inside the application (currently stored in the grade table) and add a grade-scheme class.
- Use a connection pool and add automated unit tests for the service layer.
- Future ideas from the SRS: e-mail/SMS notifications, analytics and reports, mobile companion app and a cloud-hosted database.

9. Individual Contributions



TG/2024/2095 - K.H.M. Hasaranga (Group Leader)

Role: Project Lead, Repository Manager, Core Infrastructure and Admin Module Developer

GitHub Branch: TG2095_Hasaranga (merges into dev and main)

GitHub Total commits: 131 (55 authored commits + 39 commits for pull requests)

Files Edited/Created:

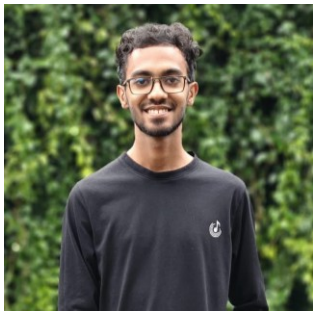
- **MainApp.java, LoginController.java** (application start-up and login screen)
- **DBConnection.java, DBTest.java** (JDBC connection and health check)
- **User, Staff, UserRole, Admin** (model hierarchy), **UserDAO / UserDAOImpl**
- **AuthService, SessionManager, PasswordUtil, AuthenticationException** (security core)
- **AdminDashboardController** and section controllers (users, students, lecturers, technical officers, courses, timetables, notices, marks, attendance, medical, profile), **StaffSectionBase, AdminDAO, AdminDb, AdminUi**
- **DataSeeder.java**, **db/schema.sql, db/seed_data.sql, db/create_app_user.sql, db/db.properties**, **pom.xml**, **.gitignore, README.md**, CSS (admin.css, style.css)
- SRS, EER diagram, class diagram, final report

Responsibilities and Contributions:

As group leader, Hasaranga created the GitHub repository, set up the Maven/JavaFX project and the branch workflow, designed the database schema and reviewed and merged the pull requests of all members. He also built the foundation that every other role depends on – the database connection, the user class hierarchy and the secure login – and completed the Admin module.

Key Functionalities Developed:

1. **Login and role routing:** BCrypt-verified login, uniform error messages, deactivated-account check and polymorphic dashboard selection (AuthService, User hierarchy).
2. **Session management:** SessionManager keeps the logged-in user for the whole session so every screen and service can check the role; logout() clears it so no data stays after logout (SEC-06).
3. **GitHub setup and workflow:** Created the GitHub repository, main and dev branches and one branch per member, set the pull-request workflow, reviewed and merged all pull requests (33 merged into dev) and resolved merge conflicts.
4. **Database creation and file structure:** Designed and wrote db/schema.sql (16 tables, keys, CHECK and UNIQUE constraints), db/seed_data.sql, db/create_app_user.sql (least-privilege user) and db/db.properties, and created the Maven project and the package/folder structure (config, model, dao, service, controller, util, fxml, css).
5. **Admin dashboard:** Create, update and deactivate users of all roles, and manage courses, timetables and notices through reusable section controllers.
6. **Documentation:** SRS, EER and class diagrams and the final report.



9.2 TG/2024/2096 - N.V.S. Himansana

Role: Lecturer Module Developer (Backend and GUI)

GitHub Branch: TG2096_Himansana

GitHub Total commits: 73 (22 authored + 1 merge)

Files Edited/Created:

- **LecturerDashboardController** and section controllers: **LecturerCoursesController, LecturerMaterialsController, LecturerMarksController, LecturerEligibilityController, LecturerAttendanceController, LecturerMedicalController, LecturerNoticesController, LecturerTimetableController, LecturerProfileController, StudentDetailsController**
- Services: **LecturerCourseService, LecturerMaterialService, LecturerMarksService, LecturerEligibilityService, LecturerAttendanceService, LecturerMedicalService, LecturerNoticeService, LecturerTimetableService, LecturerProfileService, StudentDetailsService**
- DAO interfaces and implementations for each service (LecturerMarksDAO, LecturerMaterialDAO, LecturerAttendanceDAO, ...)
- Models in model/lecture: MarkEntry, EligibilityRow, AttendanceSummaryRow, LecturerMaterial, MedicalEntry, LecturerNotice, StudentDetails, ...
- FXML screens in fxml/lecture/sections

Responsibilities and Contributions:

Himansana designed and implemented the complete Lecturer role, from the FXML screens to the database queries. Every service checks that the logged-in user is a Lecturer and that the selected course belongs to him or her before any data is read or changed.

Key Functionalities Developed:

1. **Marks management:** Add, update and delete marks for any evaluation type with CA/FINAL category and 0–100 validation.
2. **Eligibility:** Batch and individual eligibility using CA average ≥ 40% and attendance ≥ 80% (approved medicals counted).
3. **Attendance and medical views:** Batch summary and individual view of attendance and medical records of the lecturer's courses.
4. **Course materials:** Upload, modify and remove materials for assigned courses.
5. **Profile, notices, timetable, student details:** Profile update (except username and password), role-targeted notices, lecturer timetable and student detail screens.



TG/2024/2080 - I.A.S.S. Appuhami

Role: Undergraduate (Student) Module Developer

GitHub Branch: TG2080_Sudeshika

GitHub Total commits: 101 (38 authored + 1 merge)

Files Edited/Created:

1. **StudentDashboardController, HomePage** (dashboard shell and home content)
2. **StudentCoursesController, StudentAttendanceController, StudentMedicalController, StudentGradesController, StudentTimetableController, StudentNoticesController, StudentProfileController**
3. **StudentService** (GPA and summaries), **StudentDAO / StudentDAOImpl**
4. **Undergraduate, StudentRecords** models, **StudentUi** helper
5. FXML screens (student_dashboard, HomeContent, sections/*.fxml) and student.css

Responsibilities and Contributions:

Sudeshika developed everything an undergraduate sees after login. The module reads the student's own records only and lets the student change nothing except contact details and the profile picture.

Key Functionalities Developed:

1. **Dashboard and home page:** Navigation shell with summary cards for attendance, GPA and notices.
2. **Courses, timetable, notices:** Enrolled courses with materials, weekly timetable and role-targeted notices.
3. **Attendance and medicals:** Theory, practical and combined attendance per course, and the approval status of submitted medicals.
4. **Grades and GPA:** Grades, grade points, SGPA per semester and CGPA using StudentService.gpa().
5. **Profile:** Update of contact details and profile picture only (FR-07).



TG/2024/2127 - D.S.R.N. Dahanayaka

Role: Technical Officer Module Developer, Attendance and Medical Specialist

GitHub Branch: TG2127_Sandaru

GitHub Total commits: 66 (54 authored + 2 merge/other)

Files Edited/Created:

- **TechnicalOfficerDashboardController, ToUi**, section controllers: **ToAttendanceController, ToMedicalController, ToNoticesController, ToTimetableController, ToProfileController**
- Services: **AttendanceService, AttendanceCalculator, MedicalService, NoticeService, TimetableService, ProfileService, TODashboardService, ServiceSupport**
- DAOs: **AttendanceDAO, MedicalDAO, StudentQueryDAO, CourseQueryDAO, NoticeQueryDAO, TimetableQueryDAO, ProfileDAO** (+ implementations)
- Models in model/to: Attendance, AttendanceSummary, AttendanceSheetRow, MedicalRecord, ApprovalStatus, AttendanceStatus, SessionType, Course, Notice, TimetableEntry
- **Validator, FileStorageUtil, ServiceException, ValidationException**; fxml/to and to.css

Responsibilities and Contributions:

Sandaru built the Technical Officer module, which records the attendance and medical data that the eligibility rules of the whole system depend on. He also wrote the shared attendance rules, input validator and exception classes.

Key Functionalities Developed:

1. **Attendance entry:** 15-session sheets for theory and practical, 2 hours per session, with medical linking per absence.
2. **Attendance rules:** AttendanceCalculator – percentage, 80% rule and the five scenarios (including medicals).
3. **Medical management:** Submit/record medicals, upload documents, and approve or reject them with PENDING/APPROVED/REJECTED status.
4. **Validation and errors:** Validator (e-mail, phone, NIC, required, length) and friendly ServiceException messages.
5. **Notices, timetable and profile:** Read-only notices and department timetable, profile update except username and password.

10. GitHub Project Link

Repository URL: https://github.com/Hasaranga-kariyawasam/OOPP-2026-GP-05

Main branches: main (stable), dev (integration). **Member branches:** TG2095_Hasaranga, TG2096_Himansana, TG2127_Sandaru, TG2080_Sudeshika.

Repository Structure

```
OOPP-2026-GP-05/
├── pom.xml                    Maven build
├── README.md
├── db/
│   ├── schema.sql             16-table MySQL schema
│   ├── seed_data.sql          demo data
│   ├── create_app_user.sql    least-privilege DB user
│   └── db.properties          connection settings
├── docs/                      SRS, EER and class diagrams
├── uploads/                   profile pictures, medicals
└── src/main/
    ├── java/com/faculty/
    │   ├── MainApp.java
    │   ├── config/            DBConnection
    │   ├── model/             User, Staff, role packages
    │   ├── dao/               DAO interfaces + JDBC impls
    │   ├── service/           business rules
    │   ├── controller/        admin, lecture, student, to
    │   ├── exception/         custom exceptions
    │   └── util/              PasswordUtil, Validator, ...
    └── resources/
        ├── fxml/              screens of every role
        ├── css/               style sheets
        └── img/
```

11. References

1. University of Ruhuna, Faculty of Technology – ICT2132 Object Oriented Programming Practicum, Mini Project specification (2026).

2. Group 05 – Software Requirements Specification (SRS) for the Faculty Management System, 13 September 2026.

3. University Grants Commission (Sri Lanka) – UGC Commission Circular No. 12-2024 (grading scheme referred to in the project brief).

4. OpenJFX Project – JavaFX documentation. https://openjfx.io

5. Oracle – The Java Tutorials: JDBC Database Access. https://docs.oracle.com/javase/tutorial/jdbc/

6. Oracle – MySQL 8.0 Reference Manual and MySQL Connector/J documentation. https://dev.mysql.com/doc/

7. Provos, N. and Mazières, D. – A Future-Adaptable Password Scheme (bcrypt); jBCrypt library. https://www.mindrot.org/projects/jBCrypt/

8. Apache Maven Project – Maven documentation. https://maven.apache.org

9. Bloch, J. – Effective Java, 3rd Edition, Addison-Wesley, 2018.

12. Appendices

12.1 Hardware Requirements

Minimum configuration

- Processor: Intel Core i3 or AMD equivalent
- RAM: 4 GB
- Storage: 500 MB free disk space (application, database and uploads)
- Display: 1366 x 768 resolution
- Input: standard keyboard and mouse

Recommended configuration

- Processor: Intel Core i5 or AMD equivalent
- RAM: 8 GB or more
- Storage: 2 GB free space (SSD preferred)
- Display: 1920 x 1080 resolution

12.2 Software Requirements

- **Operating system:** Windows 10/11 or a recent Linux distribution
- **Java:** JDK 17 or newer
- **JavaFX:** 21.0.2 (downloaded automatically by Maven)
- **Database:** MySQL Server 8.0.16 or newer (CHECK constraints are enforced from this version); local install, XAMPP or Docker
- **Build tool:** Apache Maven 3.8+
- **IDE (optional):** IntelliJ IDEA
- **Libraries (Maven):** javafx-controls, javafx-fxml, mysql-connector-j 8.4.0, jbcrypt 0.4

12.3 Setup and Run Instructions

```
# 1. Create the database and tables (as MySQL root)
mysql -u root -p < db/schema.sql

# 2. Load the demo data
mysql -u root -p < db/seed_data.sql

# 3. (Optional, recommended) create the least-privilege application user
#    edit the password inside the script first
mysql -u root -p < db/create_app_user.sql

# 4. Put the connection details in db/db.properties
#    db.url=jdbc:mysql://localhost:3306/faculty_db
#    db.user=<mysql user>      db.password=<password>

# 5. Build and start the application
mvn clean javafx:run
```